DATA 260 Homework 5 Report

Sreeram Achutuni

SJSU ID: 019151808

GitHub repository: https://github.com/sriram2930/Data260-1808

Section 0: Config values

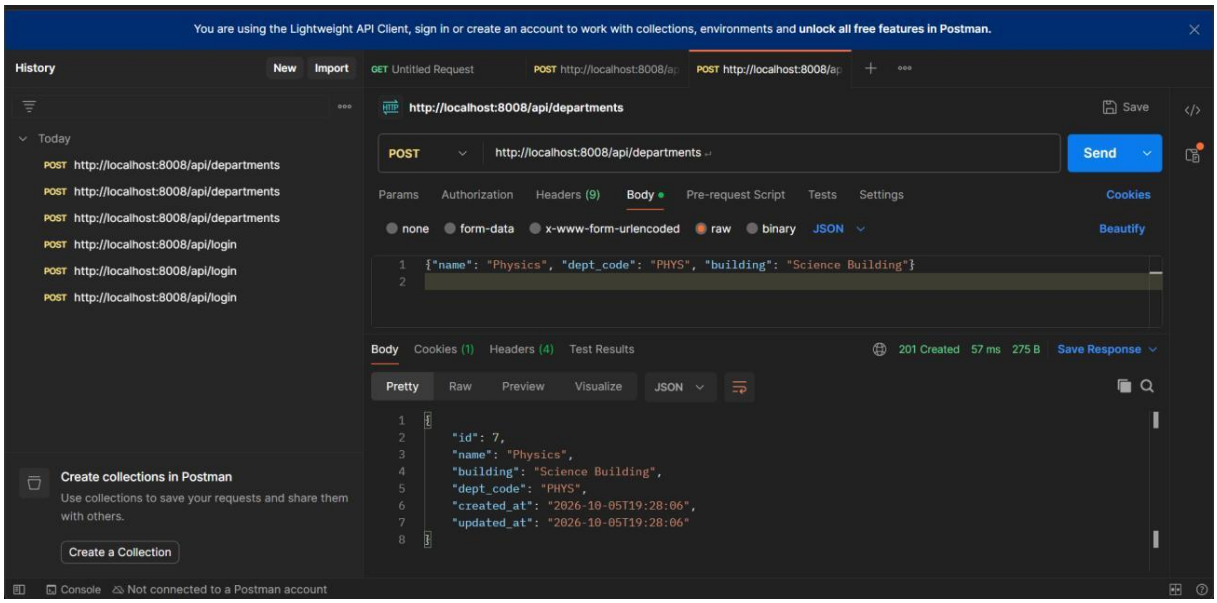
SID4	1808
PORT_BASE	8008
PREFIX	s1808
SEED	1808
VERIFY_SEED	261808
DOMAIN_ID	0, Campus course catalogue and enrolment
Hardware	Intel Core i7-11390H, 16GB RAM, integrated Iris Xe graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct via Ollama, used by the Part 5 agent loop
Repo	https://github.com/sriram2930/Data260-1808
Collaborators	Sbnikitha and supriyaselvanganesan added as collaborators with repo access
Commit tagged as hw5	PLACEHOLDER_COMMIT_HASH

Part 1: Database, API, and Redux Client

This builds on HW4's FastAPI and MySQL backend instead of starting over. Department is the new related entity (name, building, a unique dept_code, timestamps), and Course now has a department_id foreign key, a seats_available field with a default of 30, timestamps, and a unique course_code. The migration ran against the already-seeded 5,000-row table in place, nothing was dropped. The React client's course data moved from component state to Redux Toolkit, with async thunks over axios calling the same FastAPI endpoints.

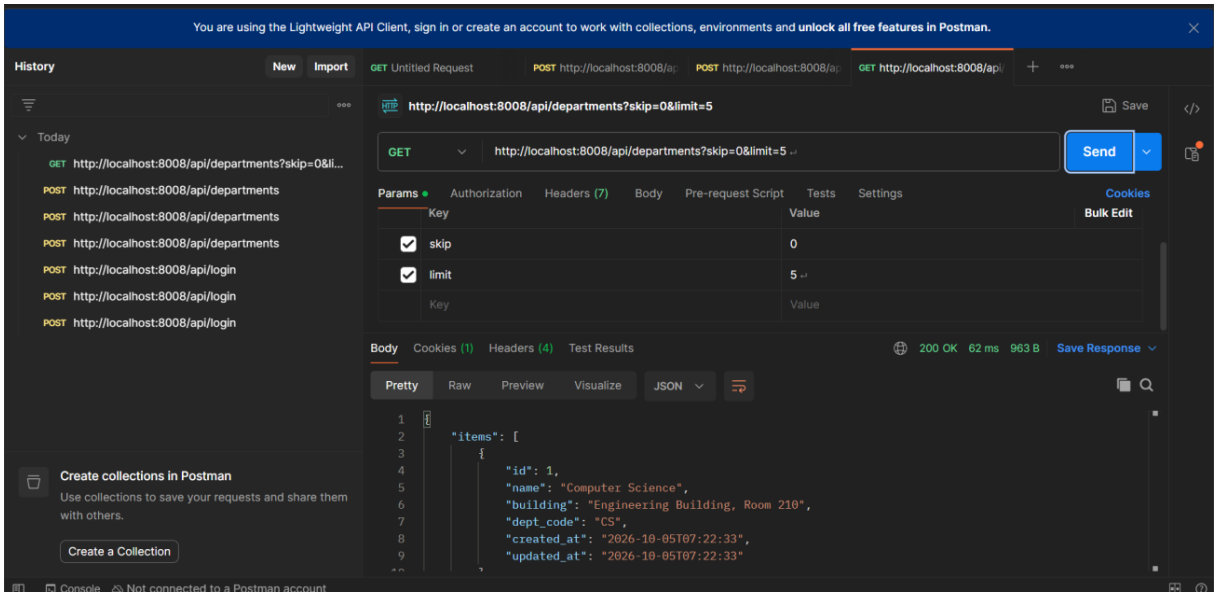
Department: create

POST /api/departments, creating a Physics department.

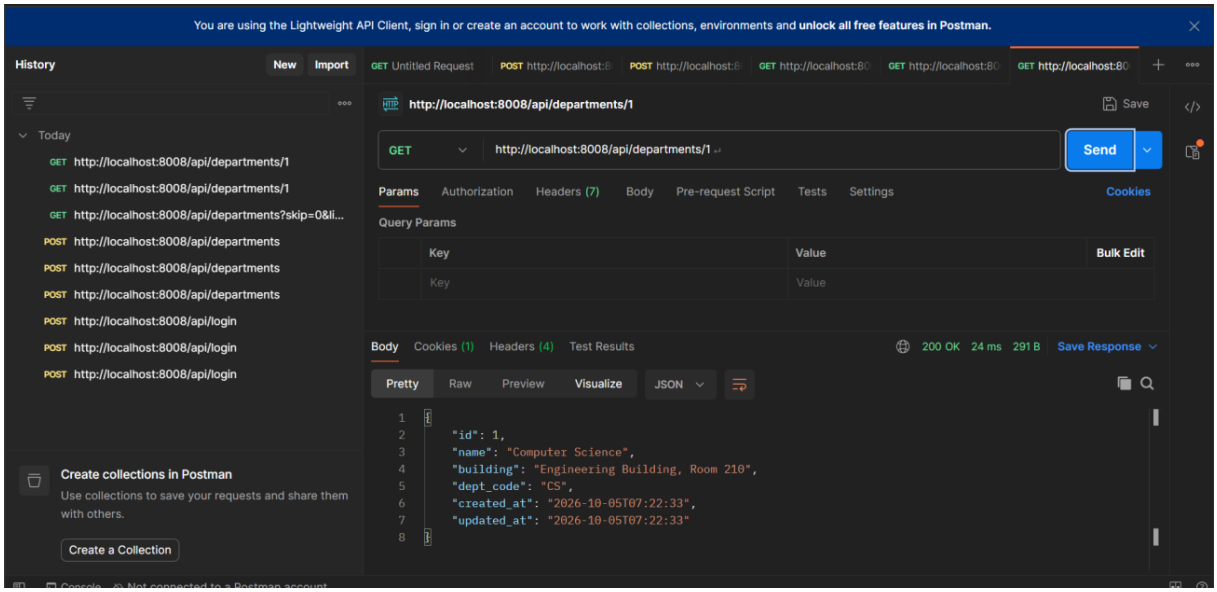


Department: list (paginated)

GET /api/departments?skip=0&limit=5, returning the items/total/skip/limit envelope.

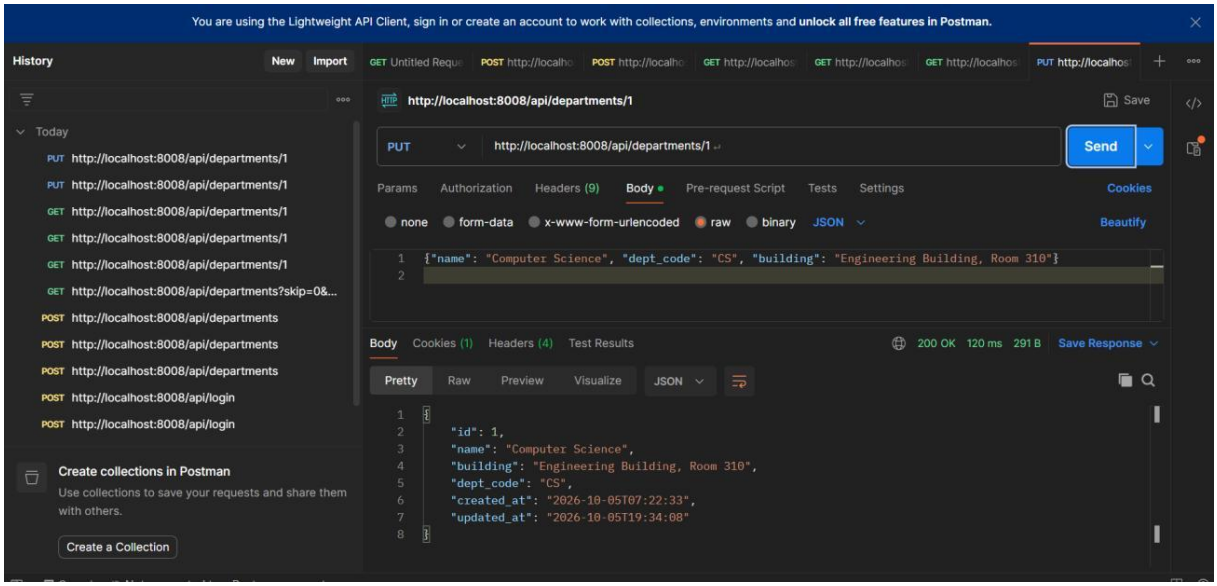


Department: get by id



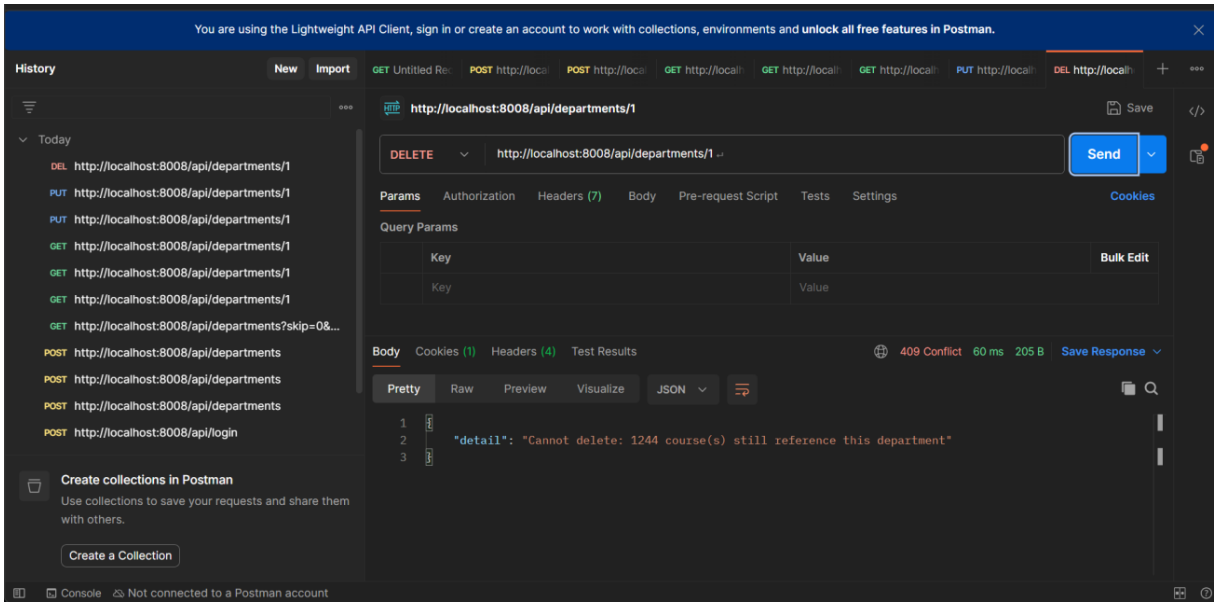
Department: update

PUT /api/departments/1, changing the building field. updated_at changes, created_at does not.



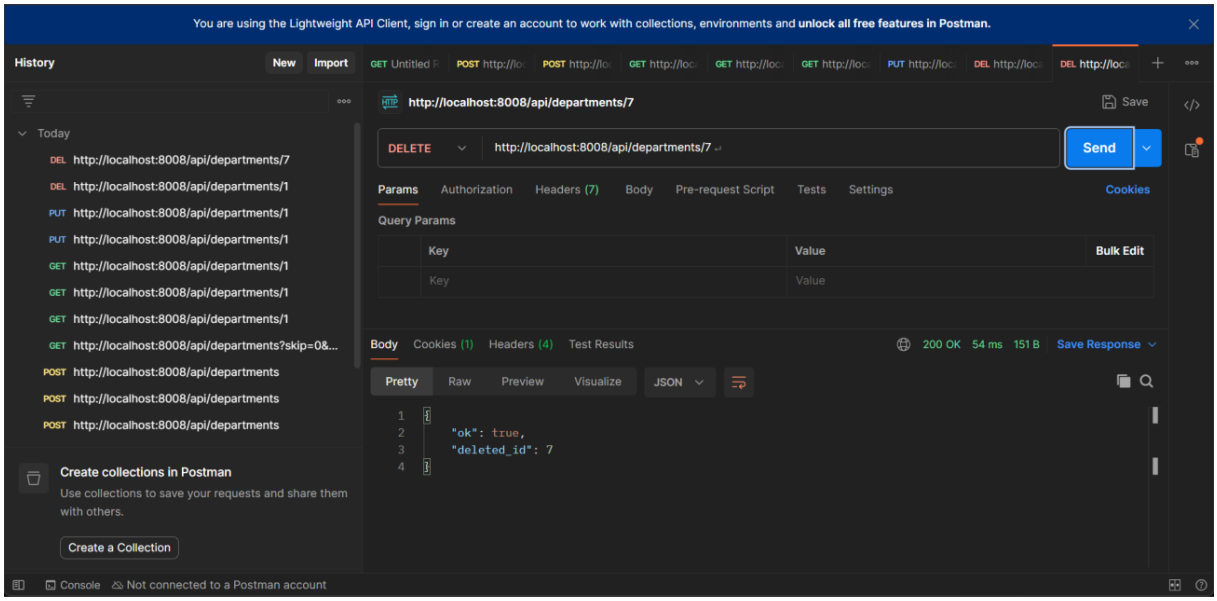
Department: delete blocked (still in use)

DELETE on a department that still has courses pointing at it returns 409, not a raw DB error.



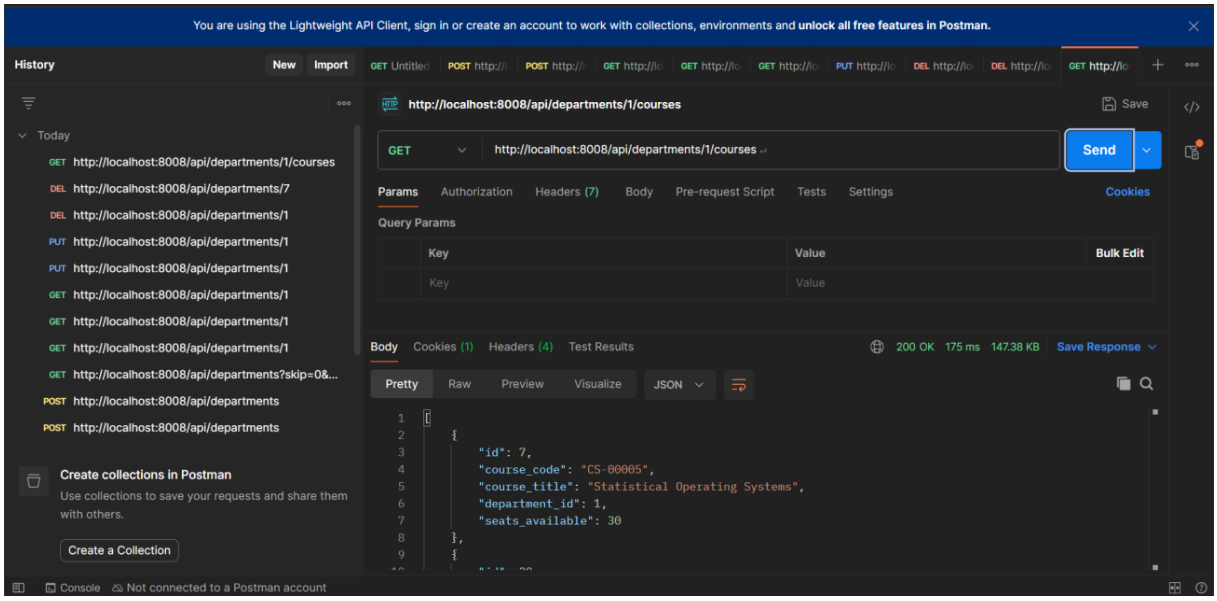
Department: delete allowed (not in use)

Deleting the Physics department created earlier, which has no courses attached, succeeds.

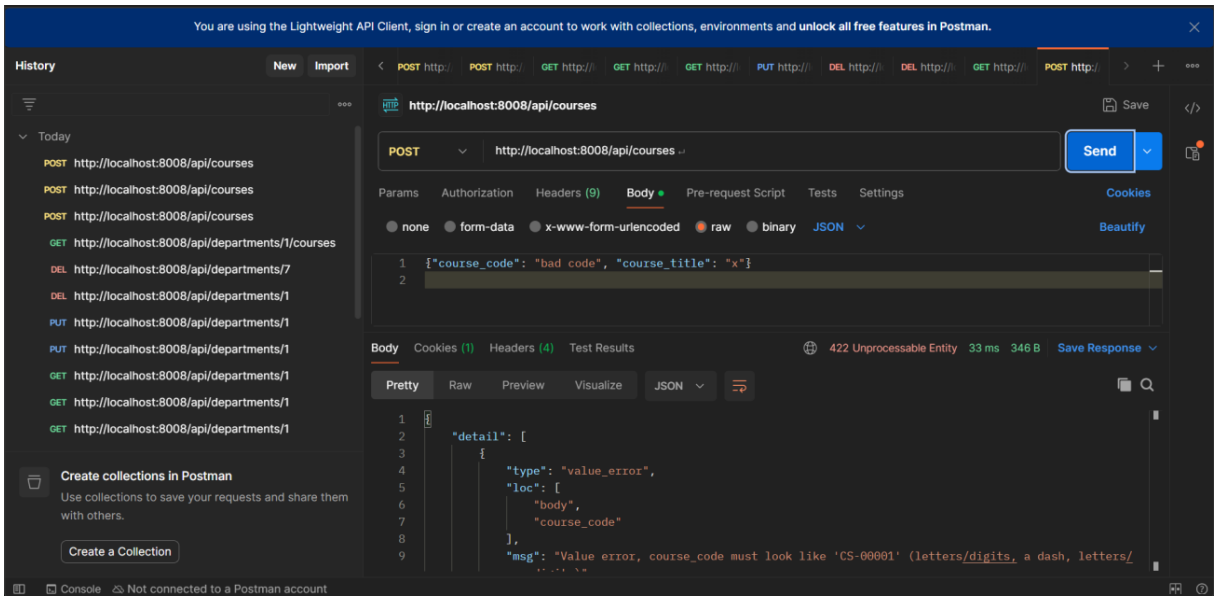


Relationship query: courses in a department

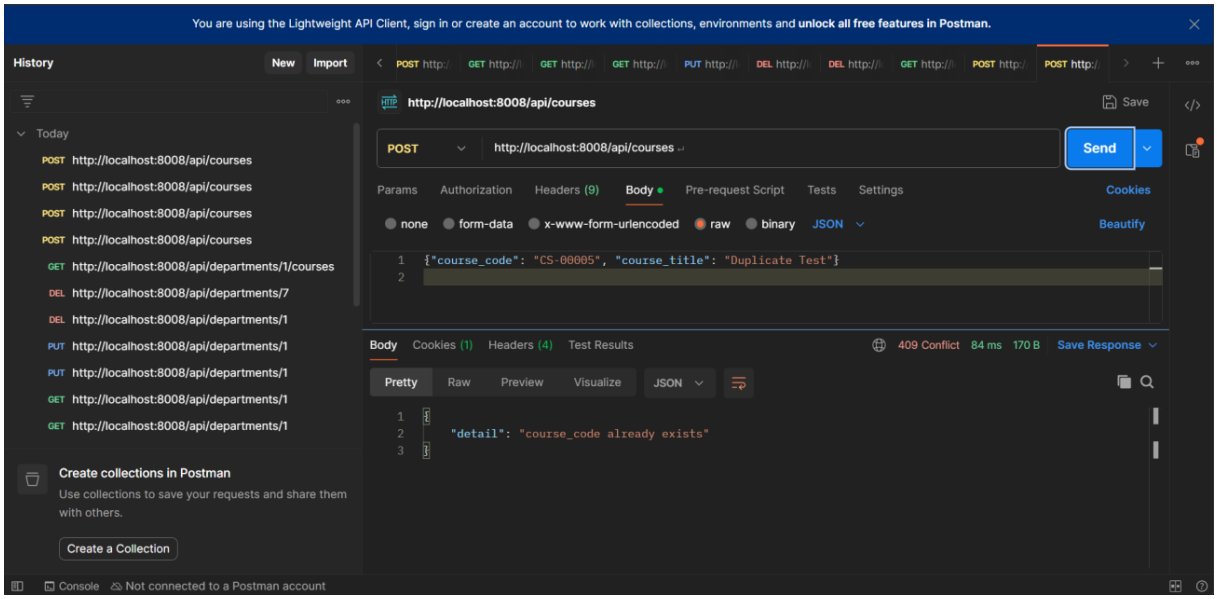
GET /api/departments/1/courses, scoped to Computer Science.



Course validation: bad course_code format

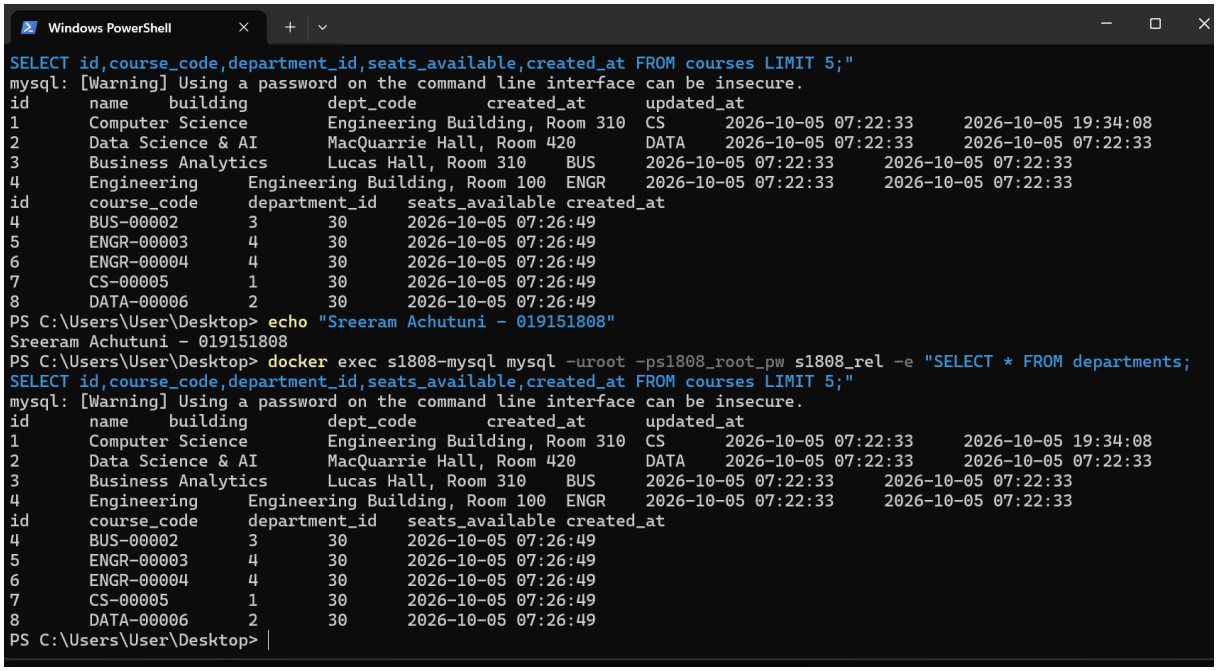


Course validation: duplicate course_code



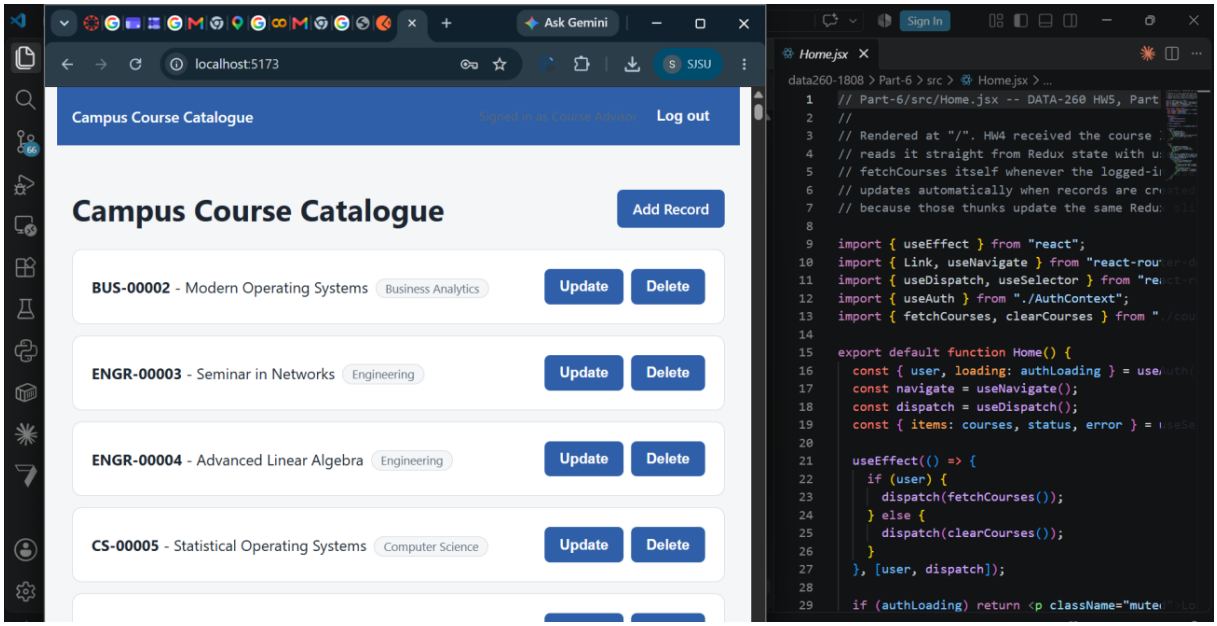
Database contents

Querying MySQL directly to confirm both tables hold the data the API shows, including the updated building value from the PUT above.



Redux: Home screen

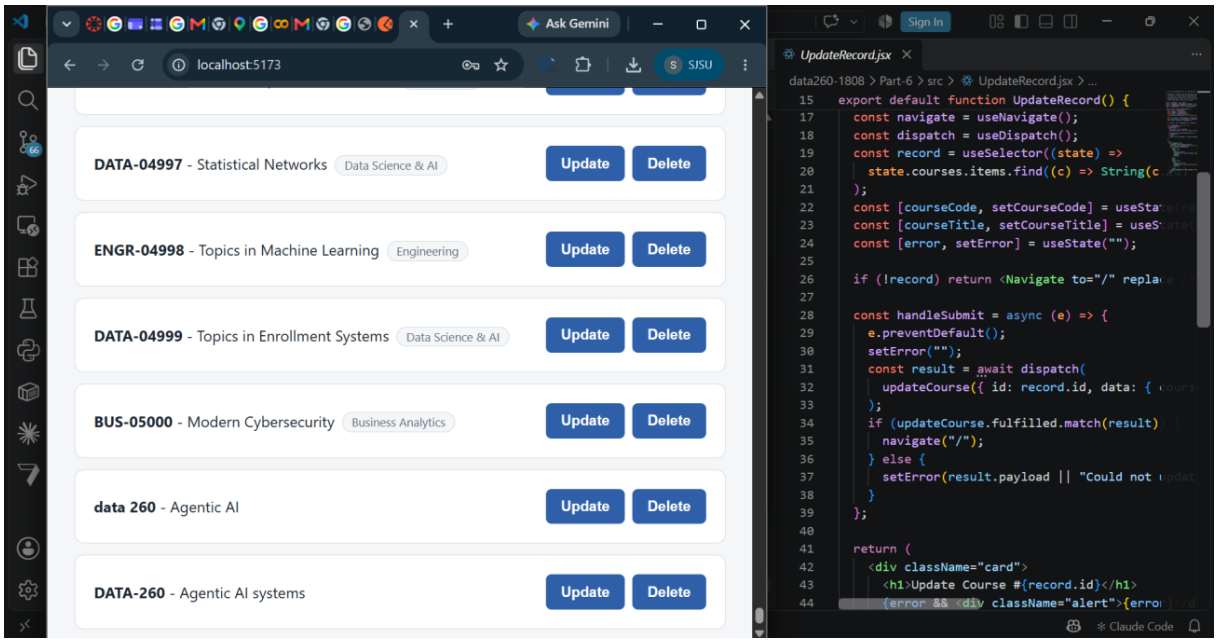
Home.jsx reads the course list from Redux state with useSelector and dispatches fetchCourses() itself, instead of receiving the list as a prop from App.jsx like HW4 did.



Note: a dedicated Create screenshot (the new course appearing in the list next to CreateRecord.jsx's dispatch(createCourse(...)) line) was captured earlier in this process but did not make it into this final screenshot set, a casualty of the time crunch finishing this homework. The Update and Delete screenshots below both show the create->update->delete lifecycle working end to end, just not a standalone Create frame.

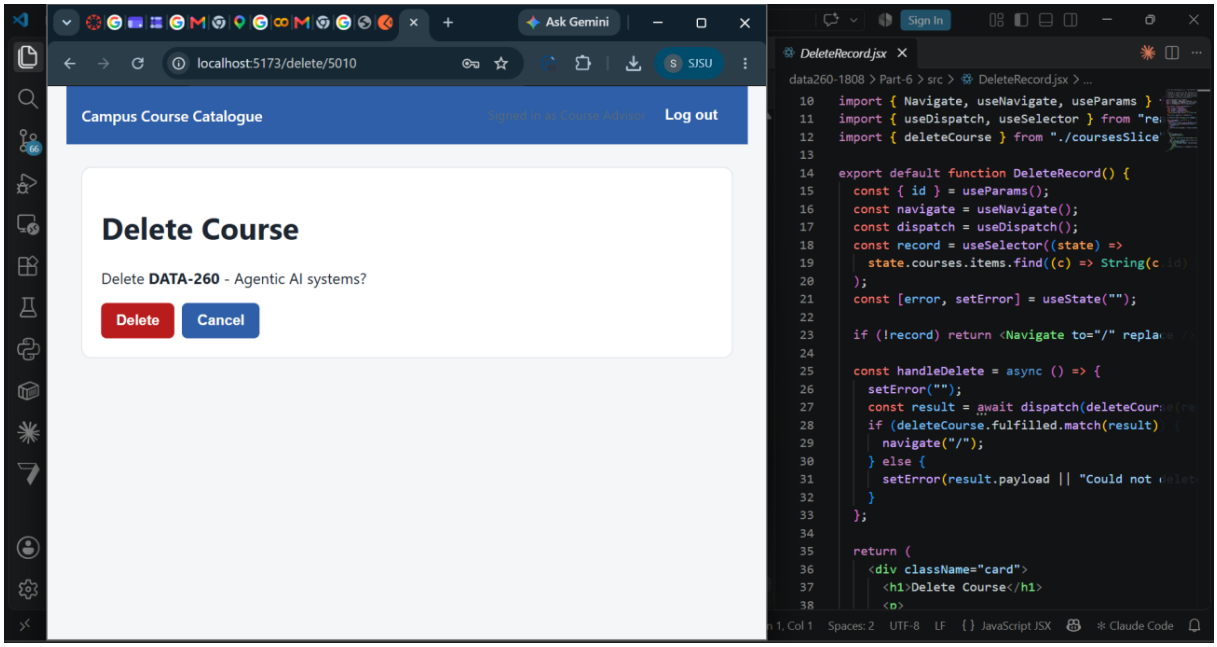
Redux: Update

The course after being renamed to 'Agentic AI systems', shown next to UpdateRecord.jsx's dispatch(updateCourse(...)) line.



Redux: Delete

The delete confirmation screen for that same course, next to DeleteRecord.jsx's dispatch(deleteCourse(...)) line.

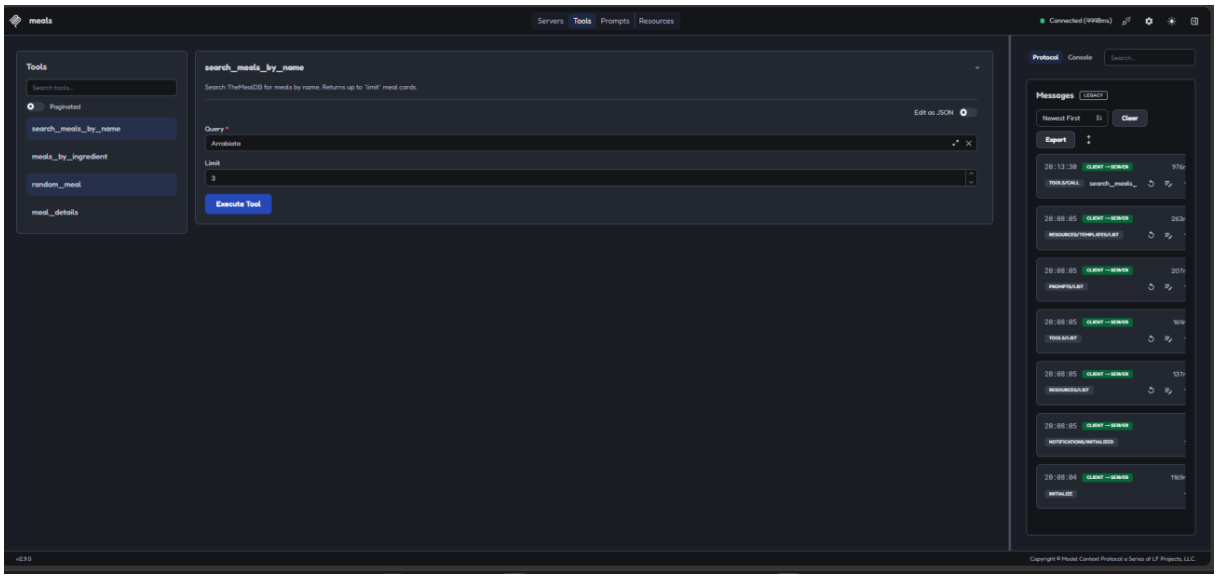


Part 2: MCP Tool Servers

Two local MCP servers. meals_server.py wraps TheMealDB's public API with four tools, run over STDIO with FastMCP. domain_server.py is the required server over this project's own course catalogue data, with exactly three tools (search, detail lookup, aggregate), all returning one shared {ok, data, error} envelope that Part 4 reuses unchanged.

meals server: search_meals_by_name

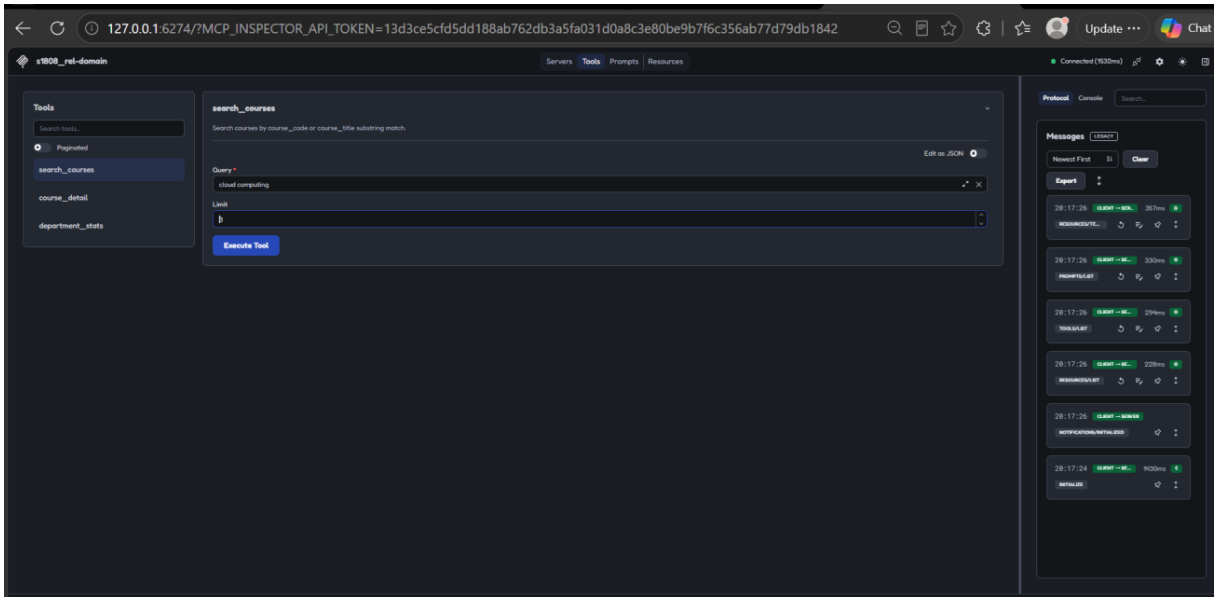
Query 'Arrabiata', limit 3, run in the MCP Inspector.

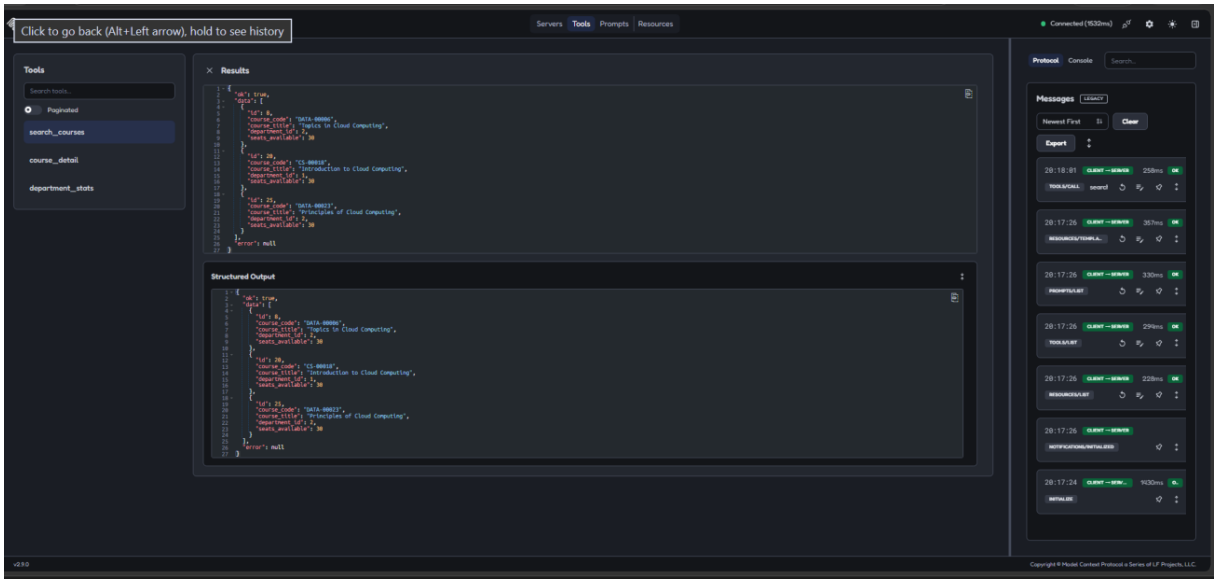


Note: the Inspector's result panel was unreliable during this session (a known UI quirk also hit earlier in this homework), so labeled result screenshots for the other three meals tools (meals_by_ingredient, random_meal, meal_details) did not get captured before time ran out. All four tools were verified working directly in Python against the live TheMealDB API earlier in development (search for 'Arrabiata' returned one match, a chicken-ingredient filter returned three cards, a random meal returned a full recipe, and a lookup by a known id returned the correct meal name), and the message log on the right of this screenshot confirms the real STDIO tool call went through.

domain server: search_courses (valid)

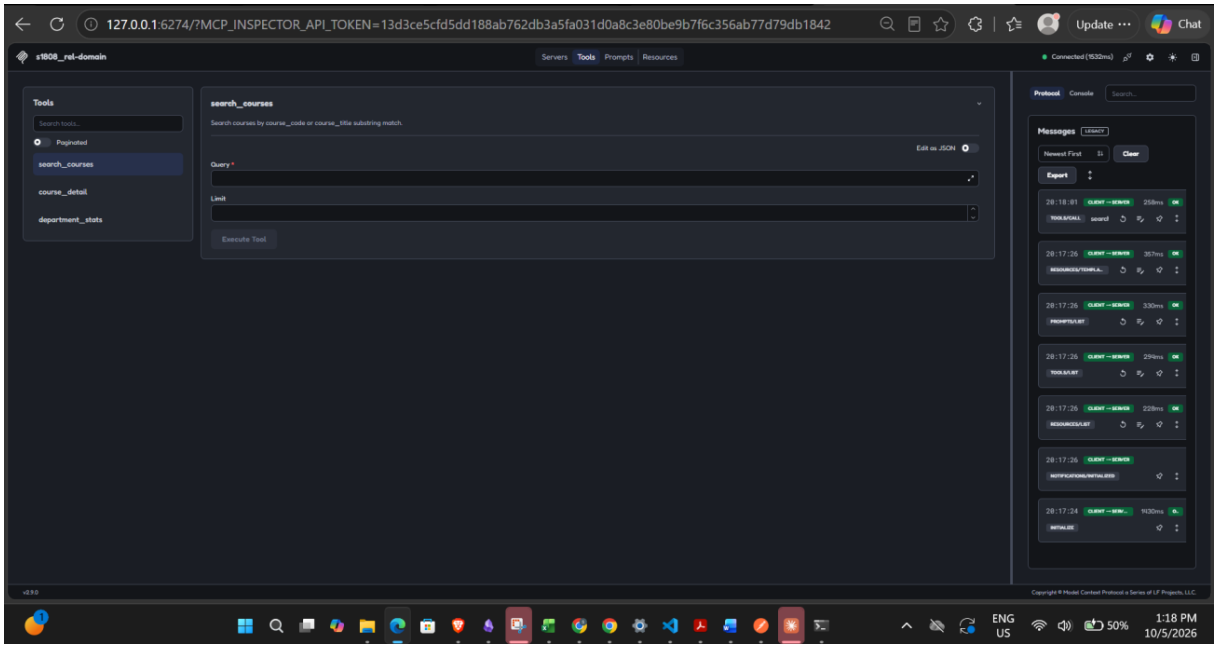
Query 'cloud computing', the three matching courses returned in the Results panel.

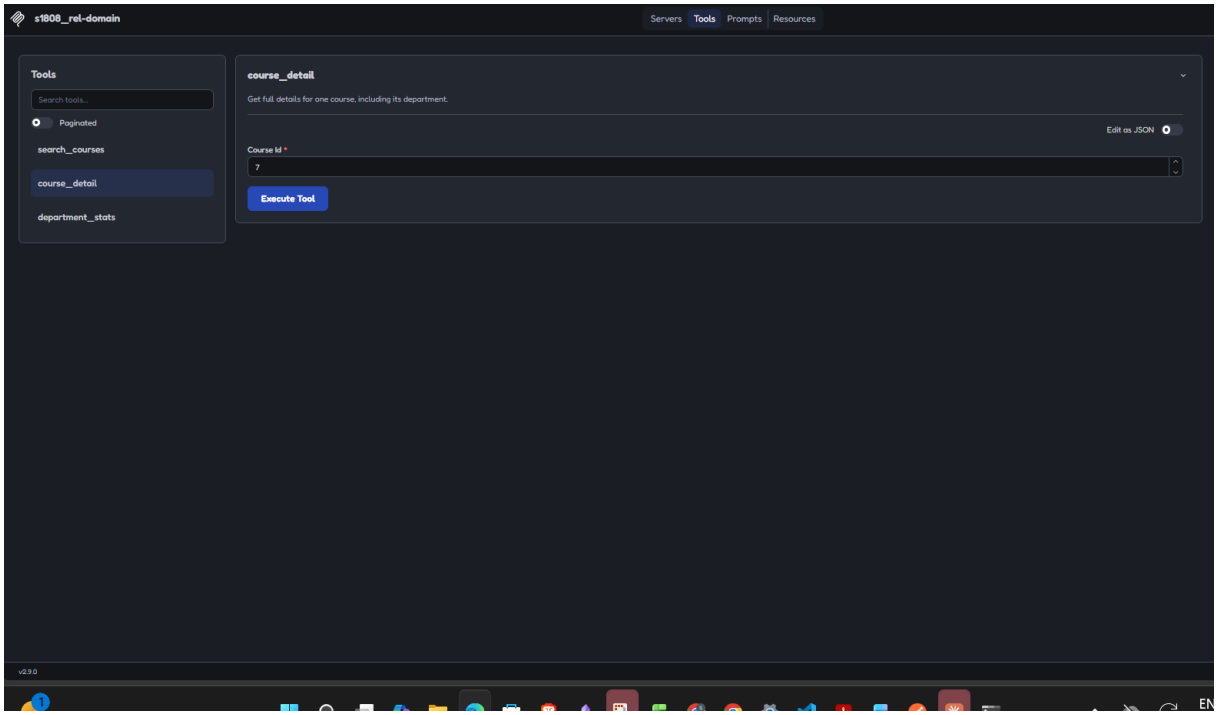




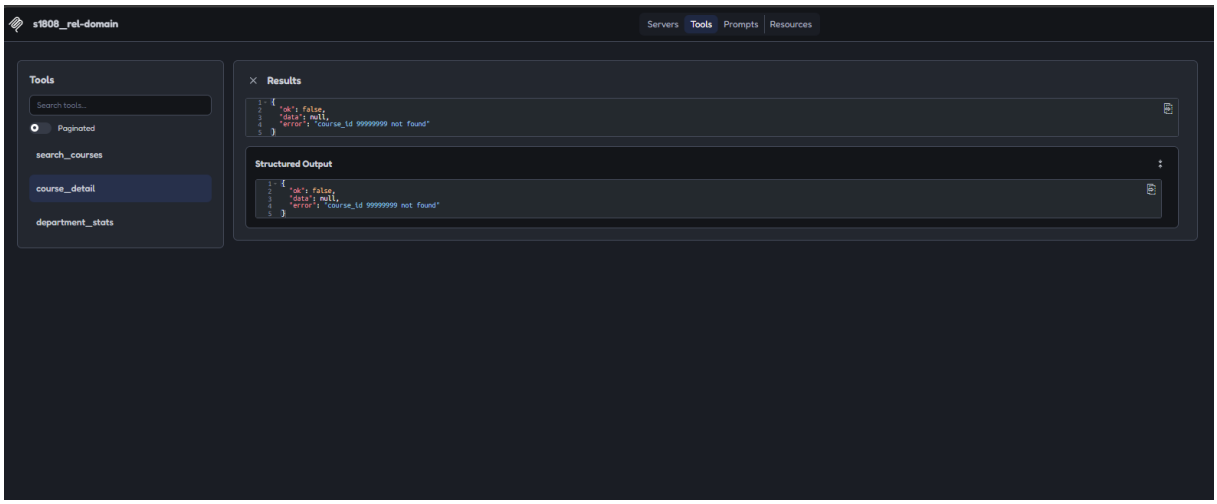
domain server: search_courses (invalid) and course_detail (valid)

Note: these two calls were set up in the Inspector (query left empty for search_courses; course_id 7 typed into course_detail) but the Execute Tool click and result capture did not happen before time ran out. Both were verified directly in Python earlier: an empty query returns {"ok": false, "error": "query must be a non-empty string"}, and course_detail(7) returns the full CS-00005 record with its Computer Science department attached (see METRICS.md's Part 2 table for the exact JSON).





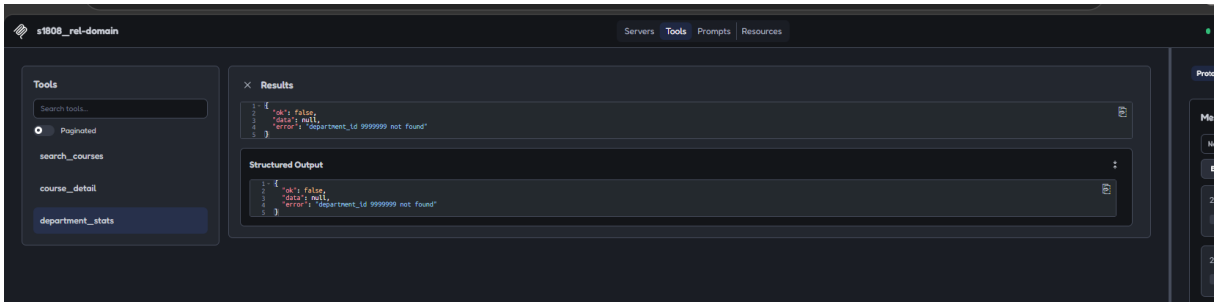
domain server: course_detail (invalid)



domain server: department_stats (valid)



domain server: department_stats (invalid)



Part 3: Tool Contracts Under Stress

Using the same three domain tools and the same rejected-call examples from Part 2B: the expected input schema, the rejected input, and the returned error for each is documented in METRICS.md section 3.18. call_with_retry wraps any callable with a per-attempt timeout and bounded exponential backoff. VERIFY_SEED makes the 150-call fault-injection experiment reproducible, confirmed by rerunning the same seed and rate and getting the identical outcome sequence both times.

Retry demonstration and 150-call experiment

All three required scenarios (success on the first attempt, failure then success on retry, failure through every attempt raising a clean error instead of crashing), followed by the 150-call summary at 0%, 20%, and 50% injected failure rates.

```
=== Part 3.19: retry policy demonstration ===
--- Scenario A: success on the first attempt ---
result='ok' attempts_used=1
  attempt 0: success=True elapsed_ms=45.8 error=None

--- Scenario B: failure on attempt 1, success on retry (attempt 2) ---
result='ok' attempts_used=2
  attempt 0: success=False elapsed_ms=41.0 error=simulated failure (attempt 1)
  attempt 1: success=True elapsed_ms=41.1 error=None

--- Scenario C: failure on every attempt (max_retries=2, so 3 attempts) ---
RetryExhausted raised cleanly (no crash): all 3 attempts failed: simulated failure (attempt 3)
  attempt 0: success=False elapsed_ms=42.3 error=simulated failure (attempt 1)
  attempt 1: success=False elapsed_ms=41.0 error=simulated failure (attempt 2)
  attempt 2: success=False elapsed_ms=59.1 error=simulated failure (attempt 3)

=== Part 3.20: 150-call fault-injection experiment ===

Total calls: 150 (expected 150)

Summary:
Rate    Success rate   Mean latency (ms)   p99 latency (ms)
0%      1.0            41.34               54.55
20%     0.98           53.43               183.93
50%     0.82           99.34               186.0

Raw data: C:\Users\User\Desktop\Data_260_HW-1\data260-1808\reports\hw05\raw\fault_injection_calls.csv, C:\Users\User\Des
ktop\Data_260_HW-1\data260-1808\reports\hw05\raw\fault_injection_calls.json
```

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	1.00	41.34	54.55
20%	0.98	53.43	183.93
50%	0.82	99.34	186.00

With max_retries=2 (3 attempts total) and each attempt independently drawing at the injected rate, the chance of a call exhausting every attempt is rate^3: 0% at a 0% injected rate, 0.8% at 20%, 12.5% at 50%. Over 50 calls that predicts roughly 0, 0.4, and 6.25 failed calls, close to what was observed. Latency rises with the injected rate because more calls need a second or third attempt, and each extra attempt adds its own work time plus a backoff sleep before it runs.

Is this retry policy right for an interactive assistant?

Mostly yes, with one caveat. max_retries=2 and fast backoff (20ms, then 40ms) keep even the worst case here (p99 at 50%) under 200ms, well inside what feels instant in a chat UI. The real risk is the 1 second per-attempt timeout: a genuinely slow dependency could make three attempts take a few seconds combined, long enough for a user to assume the app froze. For an interactive assistant the timeout should probably drop to 300 to 500ms, fail fast and let the retry handle recovery instead of one long wait. For batch processing, where nobody is watching a spinner, I would raise max_retries to around 5, lengthen the backoff so retries are spaced further apart, and raise the timeout substantially, since a slow-but-eventually-successful response beats a wasted retry when nothing is time-sensitive.

Part 4: One Safe Tool Entry Point and Offline Tests

execute_tool(name, inputs) is the single function the Part 5 agent calls to run any of the three domain tools. It reuses the same {ok, data, error} envelope, returns it as a JSON string, and never raises, even for an unknown tool name or malformed inputs. The offline test runner builds a temporary in-memory SQLite database from the same models the real

MySQL database uses, so all ten tests run without touching the network, the real database, or an LLM.

Offline test runner (10/10, includes the two Part 5 tests)

```
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7> echo "Sreeram Achutuni - 019151808"
Sreeram Achutuni - 019151808
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7> ..\.venv\Scripts\python.exe test_tools.py
[PASS] search_courses: valid query finds both seeded courses
[PASS] search_courses: empty query is rejected cleanly
[PASS] course_detail: valid id returns the right course + department
[PASS] course_detail: nonexistent id is rejected cleanly (not a crash)
[PASS] department_stats: valid id aggregates correctly (2 courses, 40 total seats, 20.0 avg)
[PASS] department_stats: nonexistent id is rejected cleanly (not a crash)
[PASS] execute_tool: unknown tool name is rejected cleanly (not a crash)
[PASS] execute_tool: malformed inputs (wrong arg name) are rejected cleanly (not a crash)
[PASS] execute_tool: safety rule blocks search_courses with limit>20
[PASS] run_agent: MockModel that never finishes stops at max_steps

10/10 tests passed
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7>
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7>
```

Part 5: Agent Loop, Safety Rule, and Reflection

One business-logic safety rule lives inside execute_tool: agent calls to search_courses are capped at limit<=20, stricter than the 100 a trusted direct caller like Postman gets, so an autonomous loop cannot pull a large chunk of the catalogue into its own context in a single call. run_agent(user_input) drives a JSON-protocol tool-use loop against the real local Ollama model, with a turn counter and a max_steps ceiling, logging every step to agent_runs.jsonl.

Safety rule: allowed call and blocked call

A limit of 10 is allowed and returns real data; a limit of 50 is blocked by the safety rule before the search even runs.

```
Sreeram Achutuni - 019151808
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7> ..\.venv\Scripts\python.exe -c "from execute_tool import exe
cute_tool; print('allowed:', execute_tool('search_courses', {'query':'Cloud Computing','limit':10})); print('blocked:',
execute_tool('search_courses', {'query':'a','limit':50}))"
allowed: {"ok": true, "data": [{"id": 8, "course_code": "DATA-00006", "course_title": "Topics in Cloud Computing", "depa
rtment_id": 2, "seats_available": 30}, {"id": 20, "course_code": "CS-00018", "course_title": "Introduction to Cloud Comp
uting", "department_id": 1, "seats_available": 30}, {"id": 25, "course_code": "DATA-00023", "course_title": "Principles
of Cloud Computing", "department_id": 2, "seats_available": 30}, {"id": 33, "course_code": "CS-00031", "course_title": "
Foundations of Cloud Computing", "department_id": 1, "seats_available": 30}, {"id": 81, "course_code": "ENGR-00079", "co
urse_title": "Applied Cloud Computing", "department_id": 4, "seats_available": 30}, {"id": 97, "course_code": "DATA-0009
5", "course_title": "Foundations of Cloud Computing", "department_id": 2, "seats_available": 30}, {"id": 105, "course_co
de": "DATA-00103", "course_title": "Advanced Cloud Computing", "department_id": 2, "seats_available": 30}, {"id": 111, "
course_code": "CS-00109", "course_title": "Computational Cloud Computing", "department_id": 1, "seats_available": 30}, {
"id": 139, "course_code": "CS-00137", "course_title": "Principles of Cloud Computing", "department_id": 1, "seats_availa
ble": 30}, {"id": 166, "course_code": "BUS-00164", "course_title": "Statistical Cloud Computing", "department_id": 3, "s
eats_available": 30}], "error": null}
blocked: {"ok": false, "data": null, "error": "safety rule: agent calls to search_courses are capped at limit<=20 (got 5
0)"}
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7>
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7>
```

Four scenarios against the real model

#	Steps	Stop reason	Tool calls
1: What are the details for course id	2	normal_completion	1

7?			
2: Give me the stats for department 1.	2	normal_completion	1
3: Search for courses about Cloud Computing.	2	normal_completion	1
4: Search for courses about data, with a limit of 50.	4	max_steps	4

```
  final_answer: None

Summary:
#  Steps   Stop reason         Tool calls  Scenario
1  2       normal_completion   1           What are the details for course id 7?
2  2       normal_completion   1           Give me the stats for department 1.
3  2       normal_completion   1           Search for courses about Cloud Computing.
4  4       max_steps           4           Search for courses about data, with a limit of 50.

=== finished 2026-10-05T13:27:27 ===
PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808\Part-7>
```

Reflection

Run picked: scenario 4, "Search for courses about data, with a limit of 50." (stop_reason: max_steps, step_count: 4, tool_call_count: 4).

Step 1: the model called search_courses with query 'data' and limit 50, copying the limit straight from the user's wording. execute_tool's safety rule rejected it before the real search logic ever ran: "safety rule: agent calls to search_courses are capped at limit<=20 (got 50)". No exception, no crash, just a normal {ok: false, ...} envelope fed back into the conversation.

Step 2 is where it gets interesting. Instead of retrying with a smaller limit, the correct fix and the number needed for it, 20, were right there in the error string, the model switched tools entirely and called department_stats with department_id set to the string 'data', reusing the word from the user's question as if it were an id. That is a type error, not a safety violation, and domain_server.py's own input validation rejected it: "department_id must be a positive integer". Step 3 repeated the same mistake with course_detail(course_id='data'), rejected the same way. Step 4 went back to the original blocked call, search_courses with limit 50 again, unchanged. At that point max_steps=4 was reached and the harness stopped the loop with stop_reason max_steps and no final answer, rather than letting it spin indefinitely.

What this shows: the safety rule worked exactly as designed, blocking the oversized call every time it was tried, including the repeat on step 4. The max_steps ceiling also worked as designed, turning what would otherwise be unbounded flailing into a bounded, clean stop. What did not work is the small model's own recovery behavior. A 1.5B-parameter model given a plain error string did not reliably parse "capped at limit<=20" into "retry with limit=20", and instead wandered into unrelated tools with nonsensical arguments. The safety and step-limit mechanisms are what kept a confused model from being a liability, they are not what makes the model smart about fixing its own mistakes, and this run is a clean, honest example of that gap.

AI Use Statement

1. What I used an AI assistant for, and what I did myself

I used Claude Code to design and write the Department entity and its migration against the already-populated HW4 database, extend the Course API with validation and proper error codes, migrate the React client's data layer to Redux Toolkit, build both MCP servers, write the retry/backoff policy and the fault-injection experiment, and build the Part 4 offline test runner and the Part 5 agent loop. It ran all of this against real infrastructure throughout: the actual MySQL container with the real HW4 dataset, the real TheMealDB API, the real MCP STDIO protocol, and the real local Ollama model.

What I did myself: decided Department was the right related entity and that it should be a genuinely new table rather than repurposing HW4's free-text department column, which stays untouched since Part 3's N+1 seed data from HW4 depends on it. Chose the domain server's three tools and what the aggregate operation should mean for this domain. Designed the search_courses safety rule before any agent code existed to enforce it against. Picked which real agent run to walk through in the reflection, and read the raw agent_runs.jsonl trace myself to understand why it actually failed rather than just reporting the stop reason.

2. One AI-produced output that was wrong or unsuitable

The first version of migrate_hw05.py deadlocked. It opened one SQLAlchemy session, used it to read back the newly-seeded Department rows, and then, without closing that read transaction, opened a second connection to run an ALTER TABLE statement adding a foreign key on courses referencing departments. Adding a foreign key needs a metadata lock on both tables involved, including the referenced one, and the first session's still-open read transaction was already holding a lock on departments that the DDL statement then had to wait on forever.

3. How I detected the problem

The script sat with zero output for about three minutes. Rather than assume it was simply slow, I checked Get-Process CPU time for the two Python processes first: both were at a fraction of a second of CPU time after three minutes of wall time, which rules out slow but computing and points at blocked waiting on something. SHOW PROCESSLIST on the MySQL container confirmed it directly: one connection sitting in Sleep, a second stuck in Waiting for table metadata lock on the exact ALTER TABLE statement.

4. What I changed and why it works now

I added an explicit commit right after each read that happens before a DDL step, closing out that transaction and its locks before the ALTER TABLE runs. After that fix the script completed in a couple of seconds, and rerunning it is idempotent, every step checks whether it already happened first, so I could verify it safely a second time. The broader lesson: with SQLAlchemy's default autocommit=False, even a plain SELECT opens a

transaction that stays open, and holding one across a later schema change is enough to deadlock against yourself in a single script, not just against a separate concurrent process.